

**INFRASTRUCTURE
AS CODE**

Terraform



What is Infrastructure as Code (IaC)?

Definition:

IaC is the practice of **managing and provisioning infrastructure** (servers, networks, databases) through **code** instead of manual steps.

Why IaC is Used:

- To **automate infrastructure setup**
 - Reduce **human errors**
 - Ensure **consistency** across environments
 - Enable **version control** of infrastructure
-

When IaC is Used:

- During **cloud setup** (AWS, Azure, GCP)
 - CI/CD pipelines** for deployment
 - Scaling infrastructure**
 - Disaster recovery** automation
-

Popular IaC Tools:

- Terraform**
 - CloudFormation (AWS)
 - Pulumi
 - Bicep (Azure)
-

Difference Between Infrastructure as Code (IaC) and Configuration Management (CM)

Feature	Infrastructure as Code (IaC)	Configuration Management (CM)
Purpose	Sets up and manages infrastructure (like servers, networks, VMs)	Manages software, updates, and configurations on existing infrastructure
Focus Area	Provisioning infrastructure	Managing the state of system/software
When Used?	When you want to create or destroy infrastructure	When you want to configure, install, or update software
Examples	Create VMs, networks, load balancers	Install NGINX, set timezones, create users
Common Tools	Terraform, Pulumi	Ansible, Chef, Puppet

Example to Understand Easily:

Imagine you're setting up a new house:

IaC (like Terraform) = Laying the **foundation, walls, rooms,** and **electric wiring**

CM (like Ansible) = Placing **furniture,** setting the **room temperature,** and **stocking the fridge**

When to Use Terraform (IaC)?

Use Terraform when you want to:

Provision infrastructure: VMs, databases, networking

Work with **cloud providers** (AWS, Azure, GCP)

Keep your infrastructure **declarative** and **version-controlled**

Automate cloud resource creation from scratch



Example:

“Create 3 EC2 instances, a VPC, and an RDS database on AWS”

When to Use Ansible (Configuration Management)?

Use Ansible when you want to:

Install and configure software (like Apache, NGINX)

Apply **security patches, update packages**

Manage **existing servers**

Do **post-provisioning tasks**



Example:

“Install Docker and Node.js on all 3 EC2 instances and start a web server”

Can Terraform and Ansible Be Used Together?

Yes! Often used **together in real projects**:

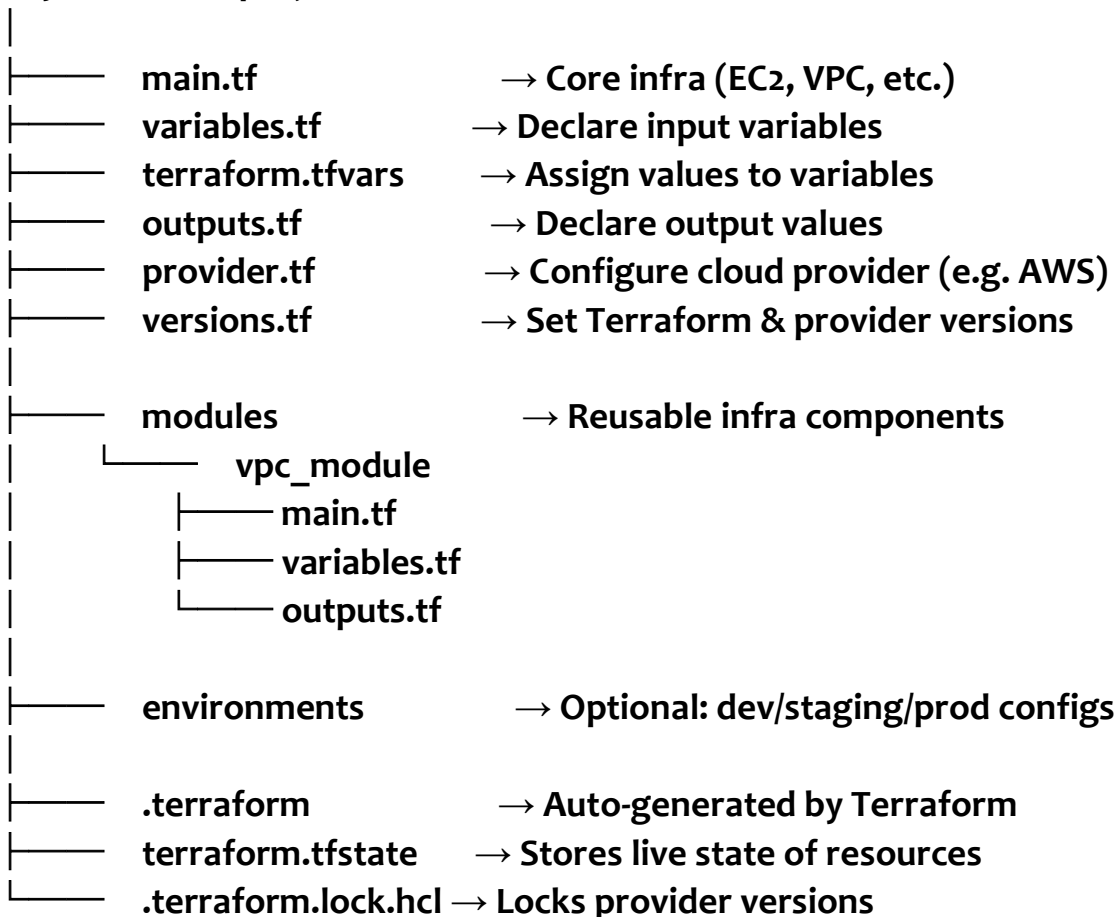
Use **Terraform** to provision cloud servers

Use **Ansible** to configure those servers after creation

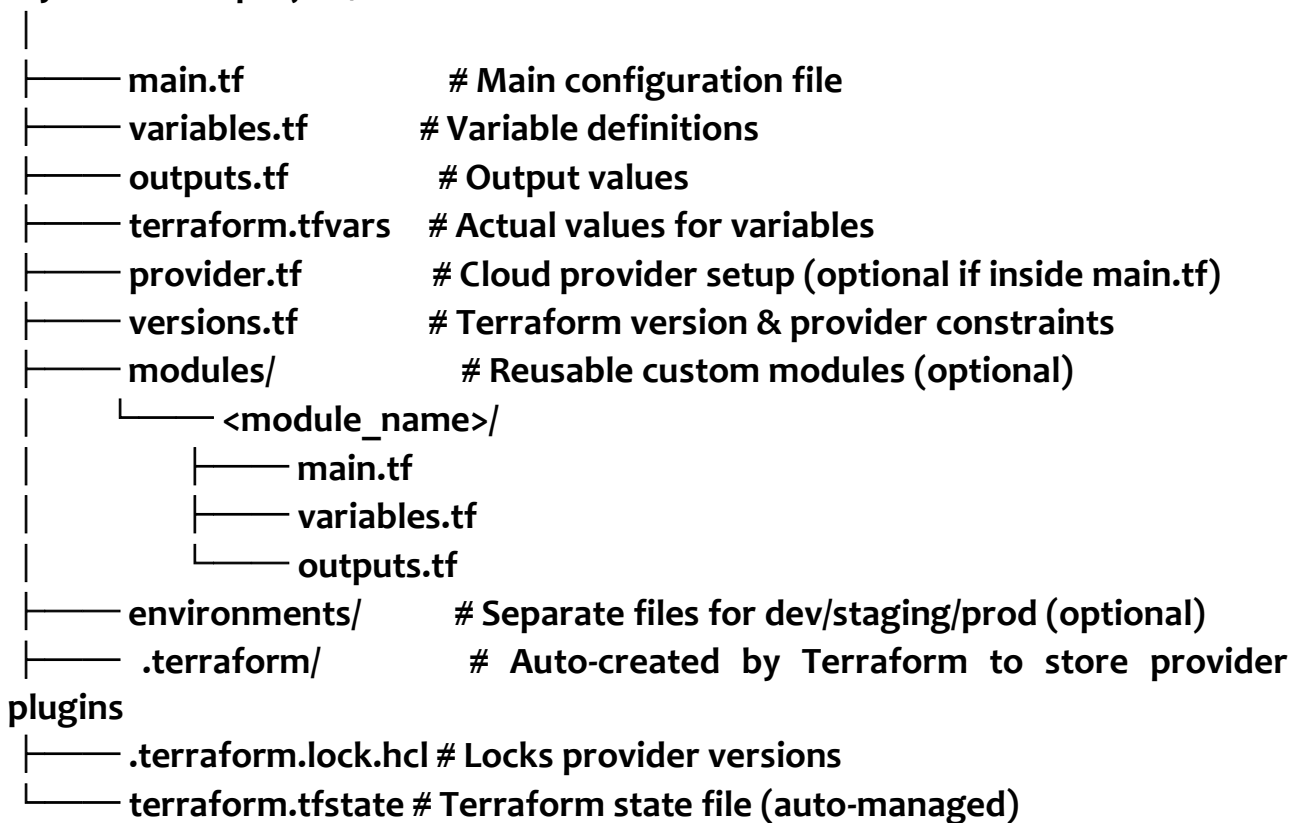
Task	Tool
Create servers, networks, databases	Terraform
Install apps, configure settings	Ansible
Use both together for full automation	 Yes

Terraform File Structure

my-terraform-project



my-terraform-project/



Explanation of Key Files and Folders

File/Folder	Purpose
main.tf	Core logic: defines infrastructure resources (EC2,VPC,etc.)
variables.tf	Declares input variables with type, default, and description
terraform.tfvars	Assigns actual values to variables (like region ="us-east-1")
outputs.tf	Defines output values to show after terraform apply (like IPs)
provider.tf	Declares cloud provider (like AWS, Azure, GCP)
versions.tf	Declares Terraform version and provider versions
modules/	Custom, reusable infrastructure components
environments/	Separate configs for dev/stage/prod (optional practice)
.terraform/	Internal cache (auto-generated, don't edit manually)
terraform.tfstate	Stores the actual state of infrastructure
.terraform.lock.hcl	Ensures consistent provider versions in teams

Terraform Architecture

CLI: Interface to interact with Terraform.

Core Engine: Reads configs, generates plan, applies changes.

Providers: Manage API calls to target platforms.

Core Terraform Commands

terraform init: Initialize directory

terraform plan: Preview changes

terraform apply: Apply changes

terraform destroy: Delete resources

terraform validate: Check syntax

terraform fmt: Format code

terraform show: View state

Providers

Example:

```
provider "aws" {  
    region = "us-east-1"  
}
```

Variables and Outputs

```
variable "region" {  
    description = "AWS region"  
    type        = string  
    default     = "us-east-1"  
}
```

terraform.tfvars

```
region = "us-west-2"
```

outputs.tf

```
output "instance_ip" {  
    value = aws_instance.web.public_ip  
}
```

State Management

Terraform uses .tfstate to track resources.

Store remotely for teams (e.g., in S3 + DynamoDB).

Protect with versioning and backups.

Modules

Modules are reusable Terraform configurations.

```
module "vpc" {  
    source = "../modules/vpc"  
    cidr_block = "10.0.0.0/16"  
}
```

Workspaces

Used to manage different environments (dev, staging, prod)

Commands: terraform workspace new dev, select, list

Best Practices

Keep state secure and remote

Use version control (Git)

Use modules for reusability

Use terraform fmt for code style

Use .tfvars for environment separation

Real-World Example

Create an EC2 Instance (main.tf)

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "web" {  
  ami           = "ami-0c55b159cbfafa1fo"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "WebServer"  
  }  
}
```



Common Errors & Debugging

State lock errors → unlock manually if needed

Provider plugin not found → re-run terraform init

Resource already exists → import or taint

Permissions issue → check IAM policies

Useful Resources

[Terraform Official Docs](#)

[Terraform Registry](#)

[Awesomme Terraform GitHub](#)
